Chương 2

CẤU TRÚC HỆ ĐIỀU HÀNH

NỘI DUNG

1. Các dịch vụ hệ điều hành
2. Giao diện hệ điều hành
3. Lời gọi hệ thống
4. Các loại lời gọi hệ thống
5. Chương trình hệ thống
6. Trình liên kết và trình nạp
7. Thiết kế và thực hiện hệ điều hành
8. Cấu trúc hệ điều hành

MỤC TIÊU

- ✓ Mô tả các dịch vụ mà hệ điều hành cung cấp cho người dùng, cho tiến trình và cho các hệ thống khác
- ✓ Giải thích được cách mà lời gọi hệ thống được sử dụng để cung cấp dịch vụ hệ điều hành
- ✓ Thảo luận về các cấu trúc khác nhau của hệ điều hành
- ✓ Giải thích hệ điều hành được cài đặt, tùy biến như thế nào và quá trình khởi động của nó

1. Các dịch vụ hệ điều hành

- Hệ điều hành cung cấp một môi trường cho việc thực hiện các chương trình và dịch vụ
- Các dịch vụ của HĐH cung cấp nhiều chức năng hữu ích cho người sử dụng:
  - Giao diện người dùng – Hầu như tất cả các HĐH đều cung cấp giao diện người dùng (user interface (UI))
    - Command-Line (CLI), Graphics User Interface (GUI), Batch
  - Thực thi chương trình - Hệ thống phải có khả năng đưa chương trình vào bộ nhớ và thực thi chương trình, kết thúc nó, theo cách bình thường hay bất thường (đưa ra lỗi)
  - Thao tác Nhập/Xuất - Chương trình đang chạy có thể yêu cầu I/O, có thể liên quan đến một tập tin hoặc một thiết bị I/O

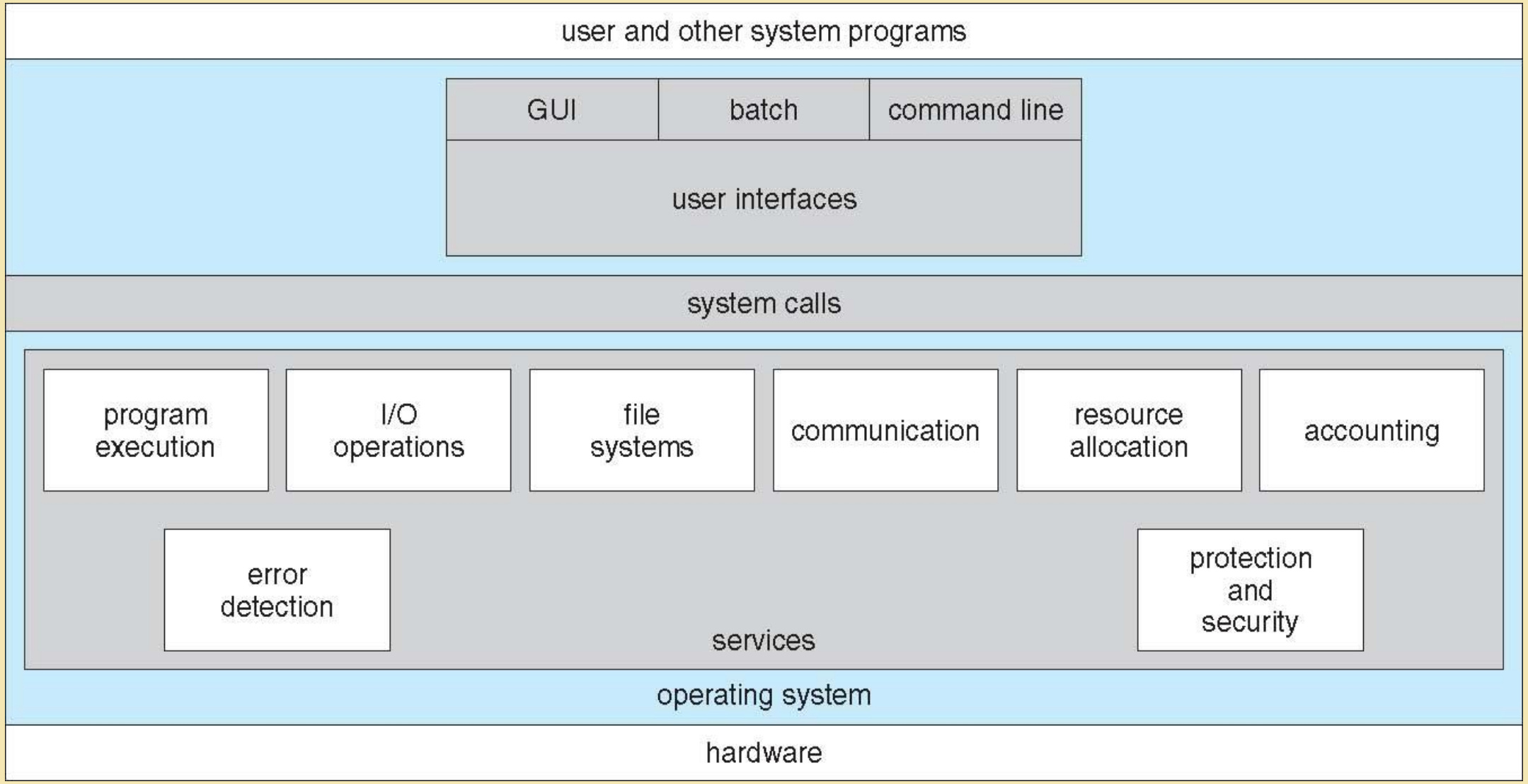
Các dịch vụ hệ điều hành

- Thao tác trên hệ thống File – Chương trình cần đọc, ghi file, thư mục; tạo, xóa, tìm kiếm, liệt kê thông tin, quản lý quyền
- Liên lạc – các tiến trình có thể trao đổi thông tin trên cùng máy tính hoặc giữa các máy tính trên mạng
  - Việc liên lạc có thể được thông qua bộ nhớ chia sẻ hoặc thông qua truyền thông điệp (gói tin được chuyển bởi hệ điều hành)
- Phát hiện lỗi
  - Xuất hiện trong CPU, bộ nhớ, thiết bị nhập/xuất, chương trình người dùng
  - Đối với từng loại lỗi, HĐH có cách giải quyết thích hợp để đảm bảo tính chính xác và nhất quán
  - Công cụ gỡ lỗi làm gia tăng khả năng cho lập trình viên và người dùng, sử dụng hệ thống hiệu quả

Các dịch vụ hệ điều hành

- Các chức năng khác của HĐH giúp đảm bảo hoạt động hiệu quả của chính nó thông qua việc chia sẻ tài nguyên
  - Cấp phát tài nguyên - Khi nhiều người dùng hoặc nhiều công việc chạy đồng thời, tài nguyên phải được cấp phát cho mỗi đối tượng
    - Nhiều loại tài nguyên - chu kỳ CPU, bộ nhớ chính, tập tin lưu trữ, thiết bị I/O
  - Tính toán – để theo dõi người dùng sử dụng bao nhiêu và loại tài nguyên nào
  - Bảo vệ và bảo mật
    - Bảo vệ: đảm bảo tất cả các truy cập đến tài nguyên hệ thống được kiểm soát
    - Bảo mật: tránh các nguy cơ từ bên ngoài tấn công hệ thống

Các dịch vụ hệ điều hành



2. Giao diện người dùng - CLI

- CLI (command-line interface) hoặc bộ dịch lệnh (command interpreter) cho phép sử dụng lệnh trực tiếp
  - Một số được thực thi trong nhân, một số bởi chương trình hệ thống
  - Trên các hệ thống có nhiều bộ dịch lệnh để lựa chọn, bộ dịch được gọi là Shells
  - Lấy lệnh từ người dùng và thực thi chúng
  - Lệnh được xây dựng sẵn hoặc tên một chương trình

Bourne Shell trong Solaris 10

Default

New Info Close Execute Bookmarks

Default | Default

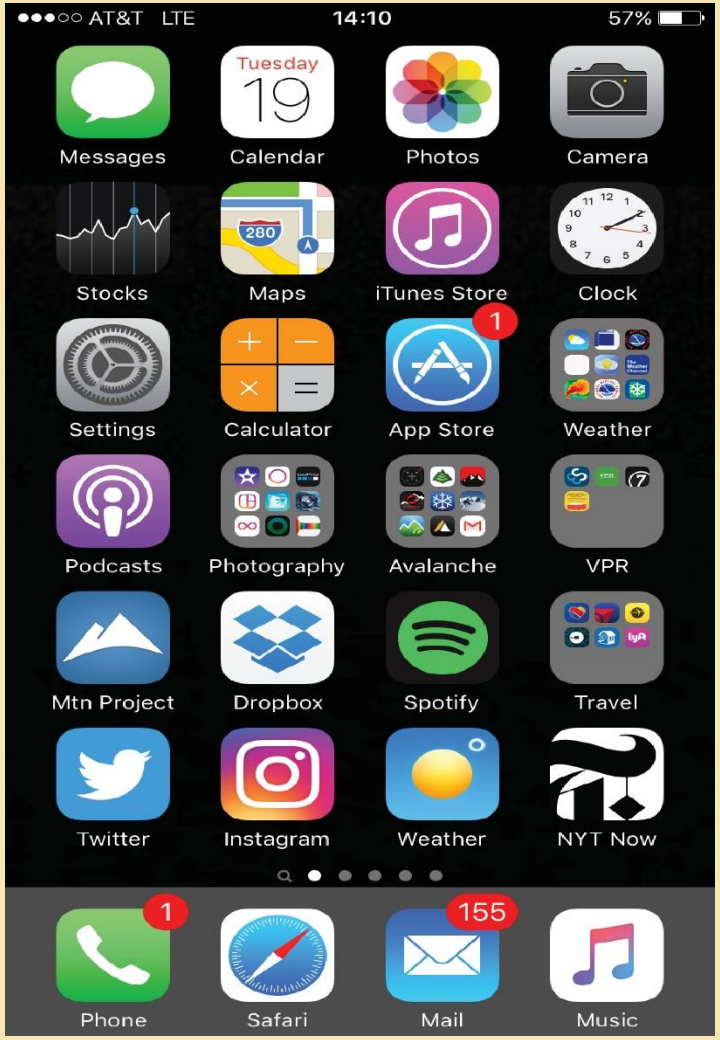
```
PBG-Mac-Pro:~ pbg$ w
15:24  up 56 mins, 2 users, load averages: 1.51 1.53 1.65
USER     TTY      FROM              LOGIN@  IDLE WHAT
pbg      console  -                14:34      50 -
pbg      s000     -                15:05       - w
PBG-Mac-Pro:~ pbg$ iostat 5
          disk0           disk1           disk10          cpu      load average
    KB/t tps  MB/s     KB/t tps  MB/s     KB/t tps  MB/s  us sy id   1m   5m   15m
   33.75 343 11.30    64.31  14  0.88    39.67   0  0.02  11  5 84  1.51 1.53 1.65
    5.27 320  1.65     0.00   0  0.00     0.00   0  0.00   4  2 94  1.39 1.51 1.65
    4.28 329  1.37     0.00   0  0.00     0.00   0  0.00   5  3 92  1.44 1.51 1.65
^C
PBG-Mac-Pro:~ pbg$ ls
Applications                    Music                           WebEx
Applications (Parallels)        Pando Packages                  config.log
Desktop                         Pictures                        getsmartdata.txt
Documents                       Public                          imp
Downloads                       Sites                           log
Dropbox                         Thumbs.db                       panda-dist
Library                         Virtual Machines                prob.txt
Movies                          Volumes                         scripts
PBG-Mac-Pro:~ pbg$ pwd
/Users/pbg
PBG-Mac-Pro:~ pbg$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1): 56 data bytes
64 bytes from 192.168.1.1: icmp_seq=0 ttl=64 time=2.257 ms
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=1.262 ms
^C
--- 192.168.1.1 ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 1.262/1.760/2.257/0.498 ms
PBG-Mac-Pro:~ pbg$
```

Giao diện người dùng - GUI

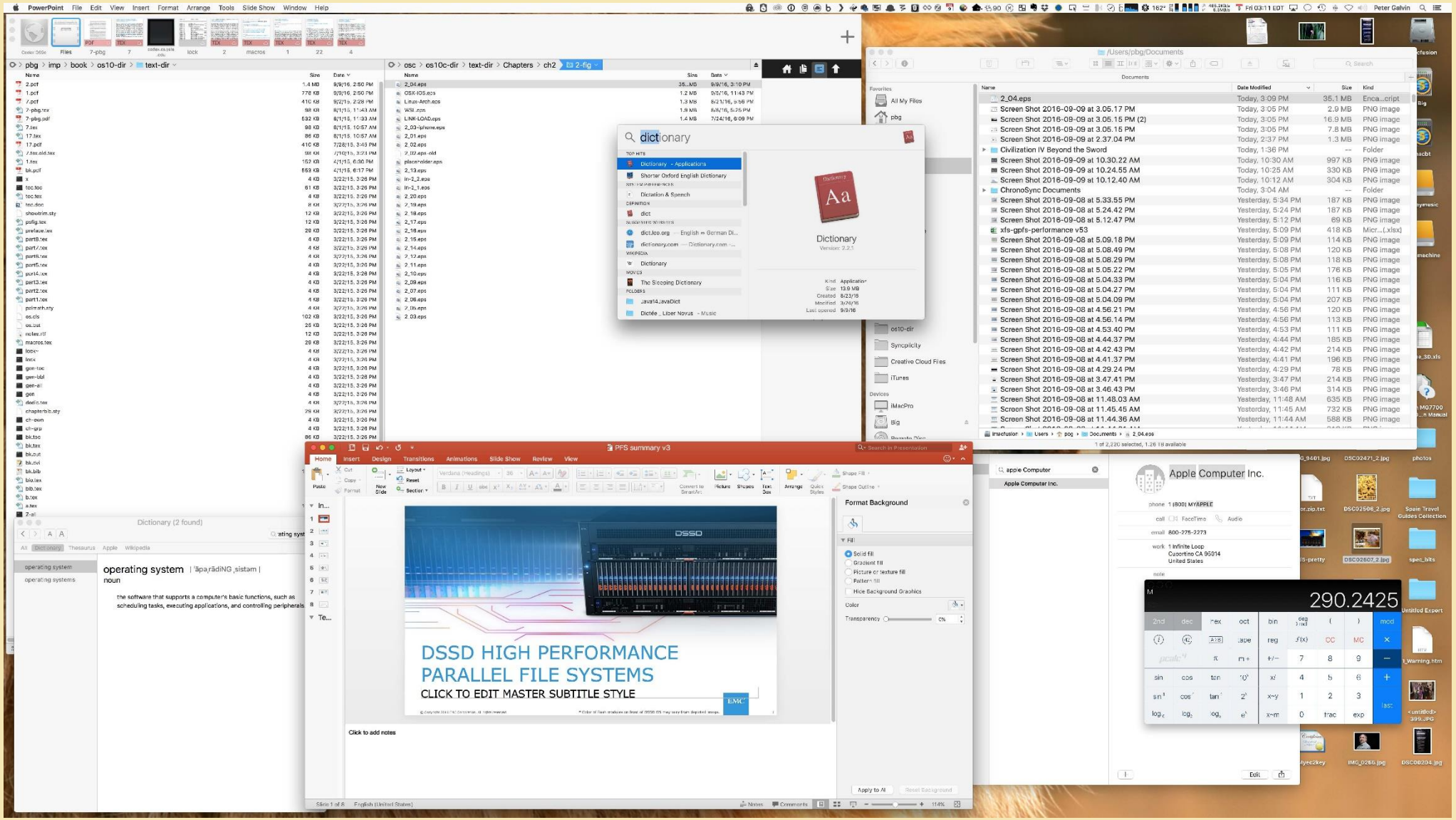
- Giao diện màn hình nền (Desktop) thân thiện
  - Thường sử dụng chuột, bàn phím và màn hình
  - Các biểu tượng (Icons) đại diện cho các tập tin, chương trình, hành động,…
  - Nút chuột khác nhau trên các đối tượng gây ra hành động khác nhau (cung cấp thông tin, lựa chọn, thực thi chương trình, mở thư mục)
  - Được phát minh tại viện nghiên cứu Xerox PARC
- Nhiều hệ thống hiện nay bao gồm cả giao diện CLI và giao diện GUI
  - Microsoft Windows sử dụng giao diện GUI có tích hợp thêm giao diện CLI là shell “Command”
  - Apple Mac OS X sử dụng giao diện GUI là "Aqua" tích hợp thêm nhân UNIX kernel và các shell có sẵn
  - Unix và Linux sử dụng giao diện CLI tích hợp thêm giao diện GUI tùy chọn cho người dùng (CDE, KDE, GNOME)

Giao diện màn hình cảm ứng (Touchscreen)

- Các thiết bị cảm ứng đòi hỏi giao diện mới
  - Không cần đến chuột
  - Thao tác và lựa chọn dựa trên cảm ứng tay
  - Sử dụng bàn phím ảo
- Điều khiển bằng giọng nói



Giao diện của Mac OS X

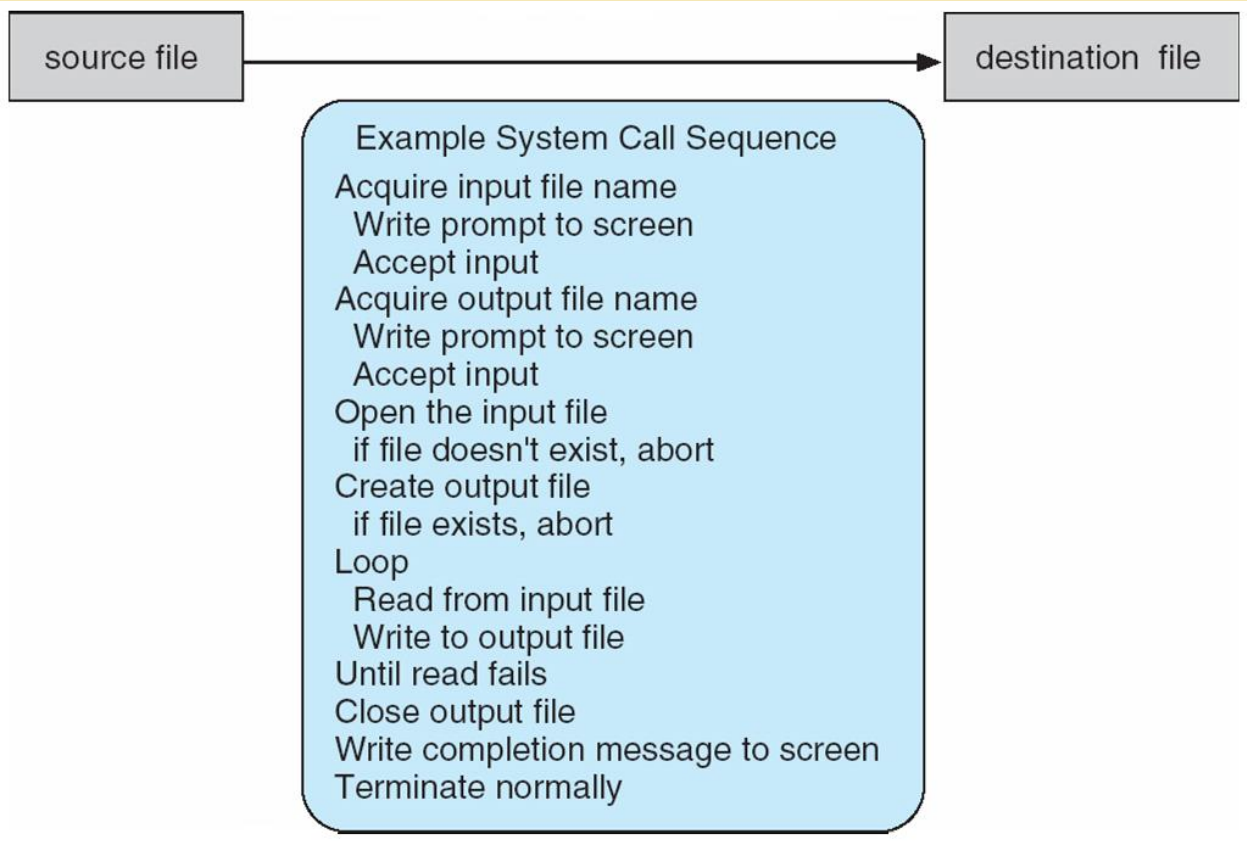


3. Lời gọi hệ thống (System calls)

- Giao diện lập trình với các dịch vụ được cung cấp bởi hệ điều hành
- Thường được viết bằng một ngôn ngữ cấp cao (C hoặc C ++)
- Được truy cập bằng các chương trình thông qua một giao diện lập trình ứng dụng cấp cao (Application Programming Interface - API) thay vì sử dụng trực tiếp lời gọi hệ thống
- Ba API phổ biến nhất là: **Win32 API** cho Windows, **POSIX API** cho các hệ thống POSIX (bao gồm hầu như tất cả các phiên bản của UNIX, Linux, và Mac OS X), và **Java API** cho các máy ảo Java (JVM)

Ví dụ về lời gọi hệ thống

- Chuỗi lời gọi hệ thống sao chép nội dung của 1 tập tin tới 1 tập tin khác



Ví dụ về API chuẩn

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

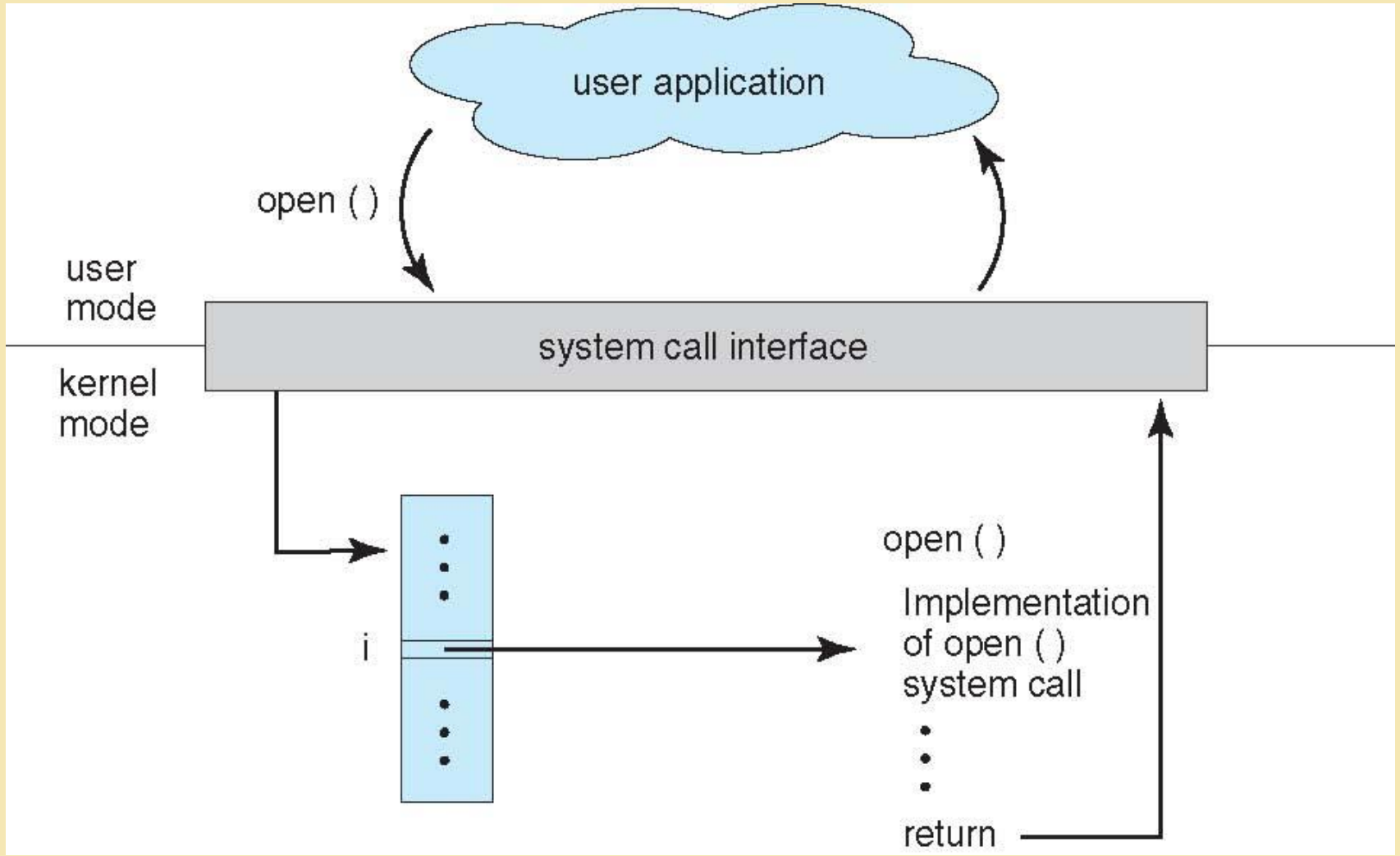
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.

Thực thi lời gọi hệ thống

- Thông thường, một con số được gắn với mỗi lời gọi hệ thống
  - Giao diện lời gọi hệ thống **(System-call interface)** duy trì một ***bảng chỉ mục*** theo các con số này
- Giao diện lời gọi hệ thống gọi lời gọi hệ thống muốn dùng trong nhân HĐH ra và trả về trạng thái của lời gọi hệ thống cùng với bất kỳ giá trị trả về khác
- Người gọi không cần biết gì về cách lời gọi hệ thống được thực hiện như thế nào
  - Chỉ cần tuân theo API và hiểu những gì HĐH sẽ làm bằng kết quả gọi
  - Hầu hết chi tiết giao tiếp HĐH với lời gọi hệ thống được ẩn từ người lập trình bằng API
    - Được quản lý bằng thư viện hỗ trợ run-time (tập các hàm được xây dựng vào thư viện bao gồm với trình biên dịch)

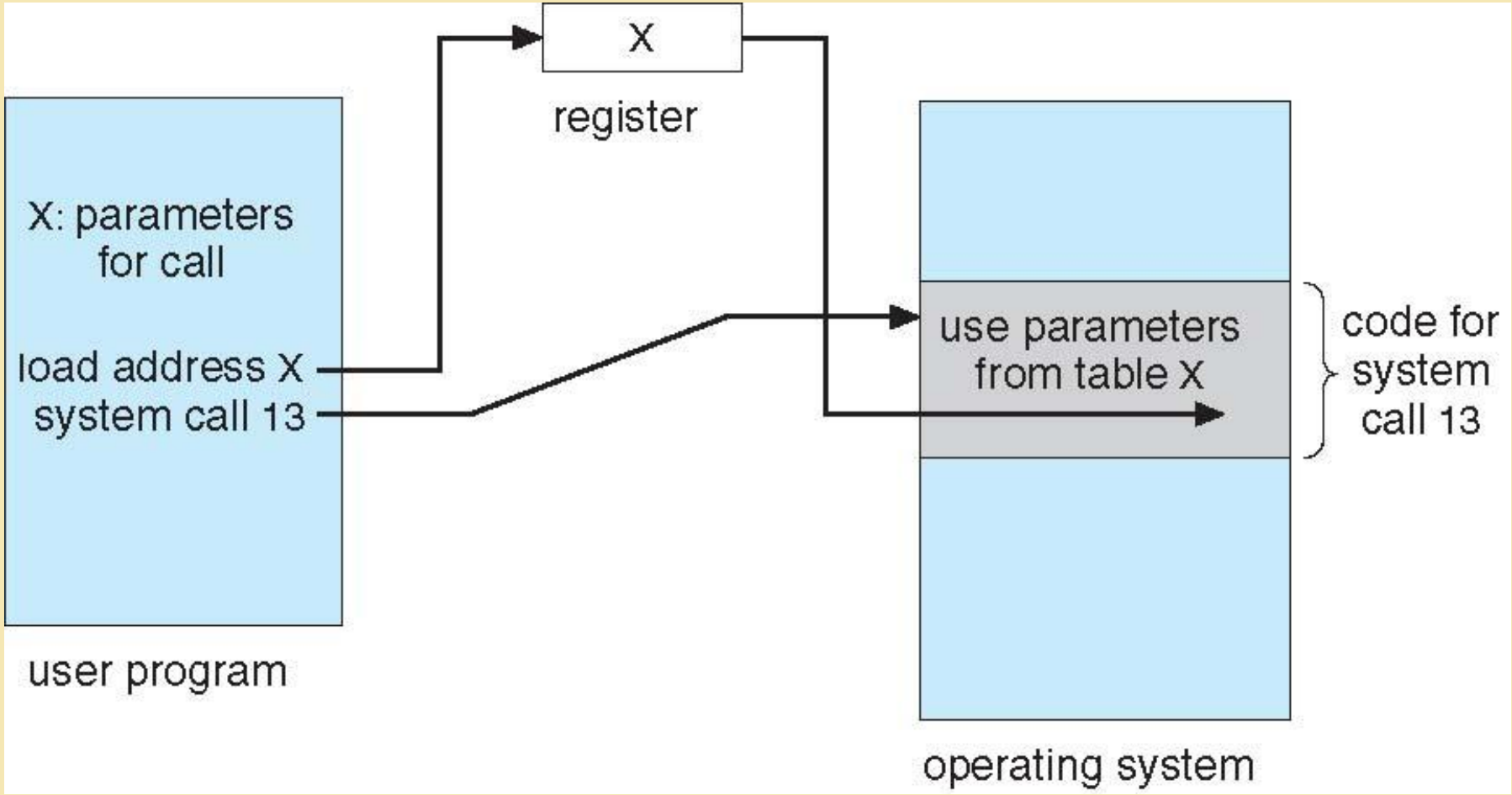
Quan hệ giữa **API – lời gọi hệ thống – OS**



Đối số trong lời gọi hệ thống

- Thường thường, có nhiều thông tin được yêu cầu hơn là đặc tính đơn giản của lời gọi hệ thống được đòi hỏi
  - Loại chính xác và số lượng thông tin khác nhau tùy theo hệ điều hành và lời gọi hệ thống
- Ba phương thức chung được sử dụng để truyền tham số tới HĐH
  - Đơn giản nhất: truyền các tham số vào các thanh ghi
    - trong một số trường hợp, có thể có nhiều tham số hơn thanh ghi
  - Tham số được lưu trữ trong một khối (block), hoặc bảng (table) trong bộ nhớ, địa chỉ của khối được truyền như một tham số vào một thanh ghi
    - Cách này được thực hiện bởi Linux và Solaris
  - Tham số được đặt, hoặc đẩy **(pushed)**, vào **stack** bởi chương trình và lấy ra khỏi **(popped)** stack bởi HĐH
  - Phương thức Block và stack không giới hạn số lượng hoặc độ dài của tham số được truyền

Truyền đối số bằng bảng



4. Các loại lời gọi hệ thống

- Điều khiển tiến trình
  - Tạo tiến trình, kết thúc tiến trình
  - Hoàn thành, hủy bỏ
  - Tải, thực thi
  - Lấy thuộc tính, thiết lập thuộc tính tiến trình
  - Thời gian chờ
  - Chờ sự kiện, chờ tín hiệu
  - Cấp phát bộ nhớ

Các loại lời gọi hệ thống (tiếp)

- Quản lý tập tin
  - Tạo, xóa
  - Mở, đóng
  - Đọc, ghi, đặt lại vị trí
  - Nhận và thiết lập các thuộc tính tập tin
- Quản lý thiết bị
  - Yêu cầu, trả lại
  - Đọc, ghi, đặt lại vị trí
  - Thiết lập thuộc tính thiết bị

Các loại lời gọi hệ thống (tiếp)

- Duy trì thông tin
  - Tiếp nhận thời gian và ngày, thiết lập thời gian và ngày
  - Tiếp nhận dữ liệu hệ thống, thiết lập dữ liệu hệ thống
  - Nhận và thiết lập tiến trình, tập tin và thuộc tính thiết bị
- Liên lạc
  - Tạo, xóa kết nối liên lạc
  - Gởi, nhận tin theo cơ chế **message passing**
  - Tạo và truy cập vào bộ nhớ theo cơ chế **Shared-memory**
  - Chuyển đổi trạng thái thông tin

Các loại lời gọi hệ thống (tiếp)

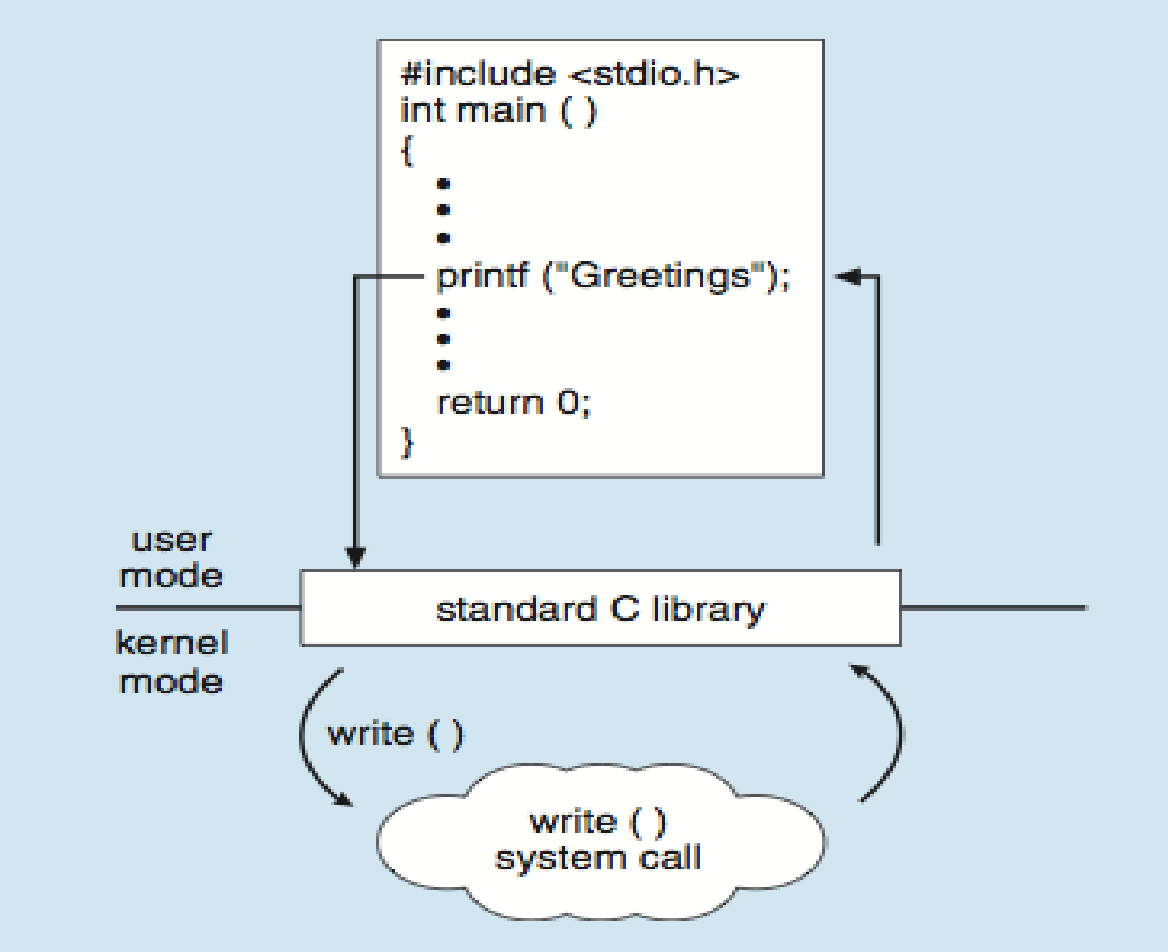
- Bảo vệ
  - Điều khiển truy cập tài nguyên
  - Nhận và thiết lập quyền truy cập
  - Cho phép và cấm người dùng truy cập

Lời gọi hệ thống trong Windows và Linux

	Windows	Unix
Process Control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File Manipulation	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Device Manipulation	SetConsoleMode()	ioctl()
	ReadConsole()	read()
	WriteConsole()	write()
Information Maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()
Communication	CreatePipe()	pipe()
	CreateFileMapping()	shmget()
	MapViewOfFile()	mmap()
Protection	SetFileSecurity()	chmod()
	InitlializeSecurityDescriptor()	umask()
	SetSecurityDescriptorGroup()	chown()

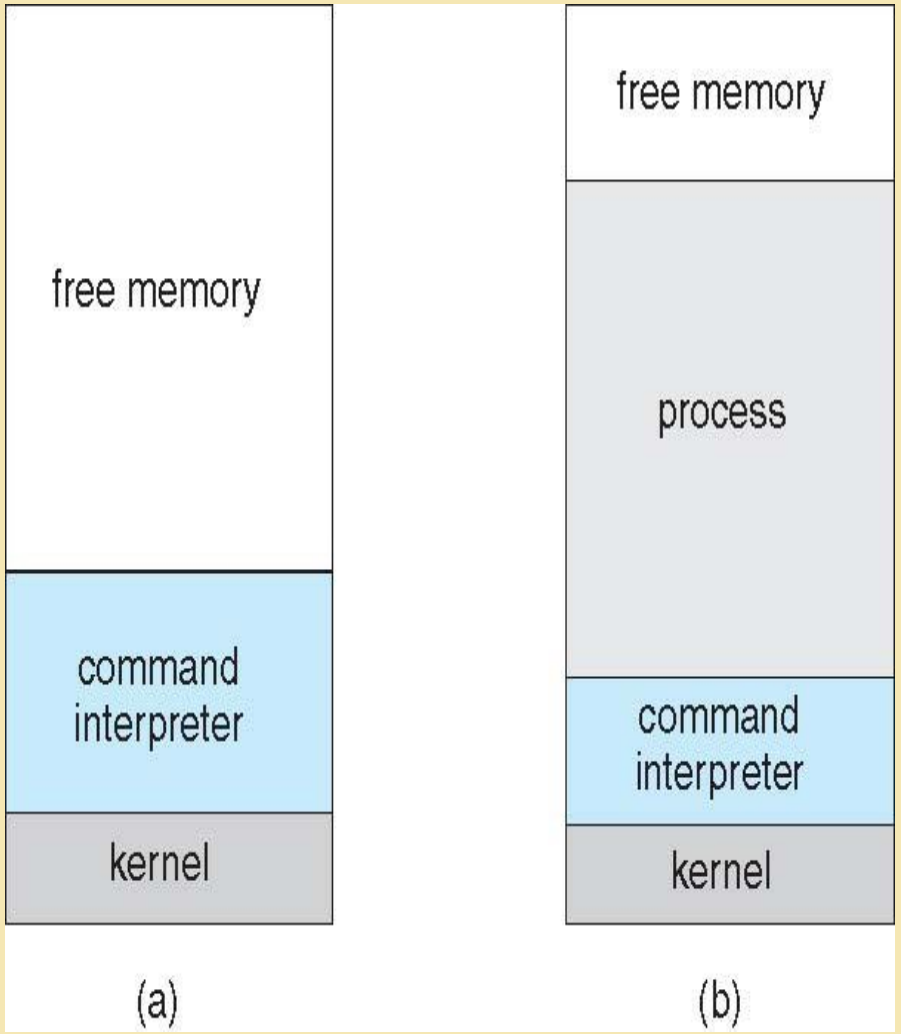
Thư viện C chuẩn

- Chương trình C gọi hàm thư viện ***printf()***, hàm thực hiện lời gọi hệ thống ***write()***



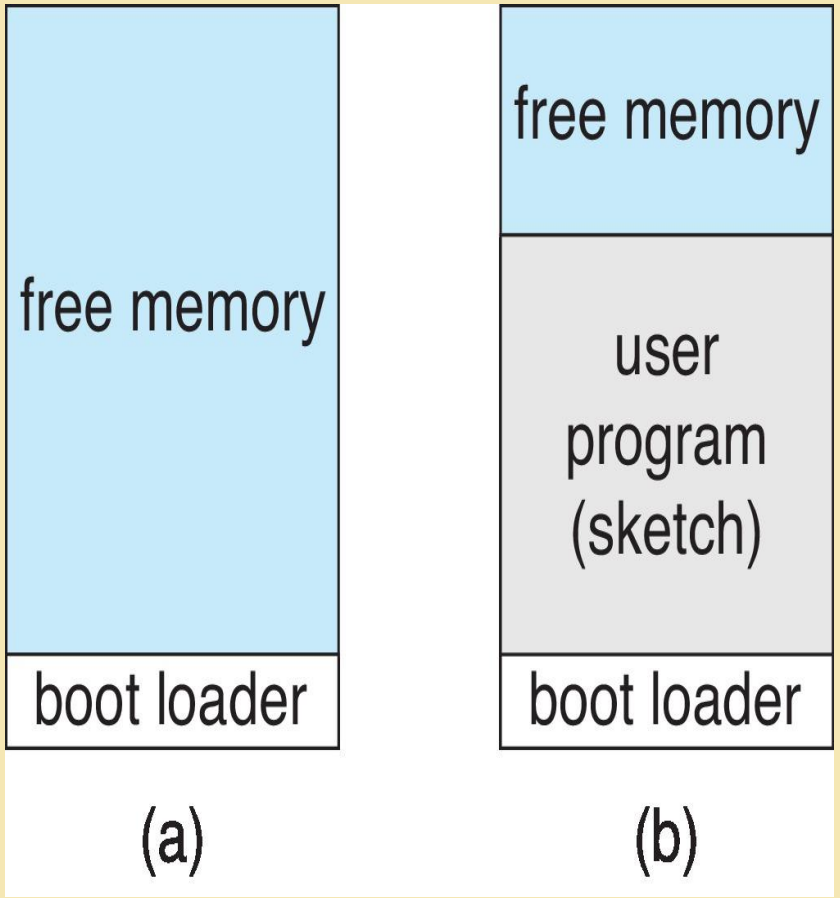
Ví dụ: MS-DOS

- Đơn nhiệm
- Shell được gọi khi hệ thống khởi động
- Phương thức chạy chương trình đơn giản
  - Không có tiến trình được tạo
- Không gian nhớ duy nhất
- Tải chương trình vào bộ nhớ, ghi đè lên tất cả trừ hạt nhân
- Thoát chương trình → Shell được nạp lại



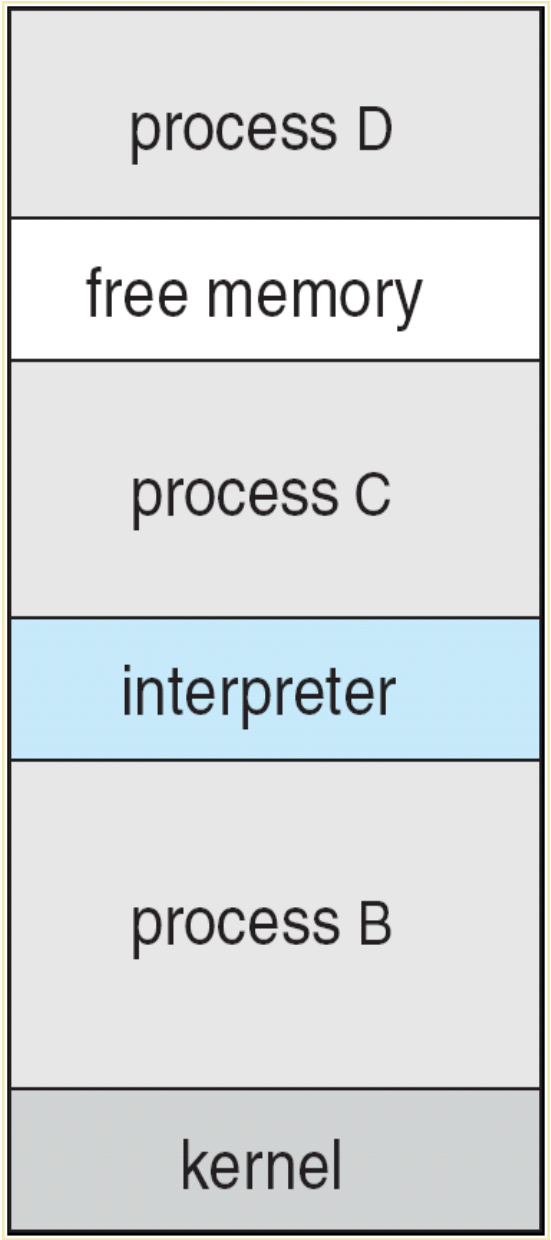
Ví dụ: Arduino

- Đơn nhiệm
- Không có hệ điều hành
- Các chương trình (sketch) được tải qua USB vào bộ nhớ flash
- Không gian bộ nhớ đơn
- Bộ nạp khởi động tải chương trình
- Thoát chương trình → Shell được nạp lại



Ví dụ: FreeBSD

- Biến thể của Unix
- Đa nhiệm
- Khi người dùng đăng nhập → gọi Shell của người dùng chọn
- Shell thực thi lời gọi hệ thống ***fork()*** để tạo tiến trình
  - Thực thi ***exec()*** để nạp chương trình vào tiến trình
  - Shell chờ tiến trình kết thúc hoặc tiếp tục với các lệnh khác của người dùng
- Khi tiến trình thoát:
  - Code=0 : không có lỗi
  - Code>0 : có lỗi



5. Chương trình hệ thống

- Chương trình hệ thống cung cấp một môi trường thuận lợi cho việc phát triển chương trình và thực thi. Chúng có thể được chia thành
  - Quản lý File
  - Chương trình quản lý trạng thái thông tin
  - Cập nhật File
  - Ngôn ngữ lập trình
  - Chương trình tải và thực thi
  - Liên lạc
  - Dịch vụ nền
  - Chương trình ứng dụng

Chương trình hệ thống

- Quản lý File – Tạo, xóa, sao chép, đổi tên, in ấn, kết xuất, liệt kê và các thao tác khác trên tập tin và thư mục
- Chương trình quản lý trạng thái thông tin
  - Một số yêu cầu hệ thống về ngày, giờ, số lượng bộ nhớ có sẵn, không gian đĩa, số lượng người dùng
  - Một số khác cung cấp hiệu suất, việc đăng nhập, và các trình gỡ rối
  - Các chương trình này định dạng và xuất thông tin ra màn hình hoặc thiết bị xuất khác
  - Một số hệ thống thực thi **registry** – đối tượng được sử dụng để lưu trữ và lấy thông tin cấu hình

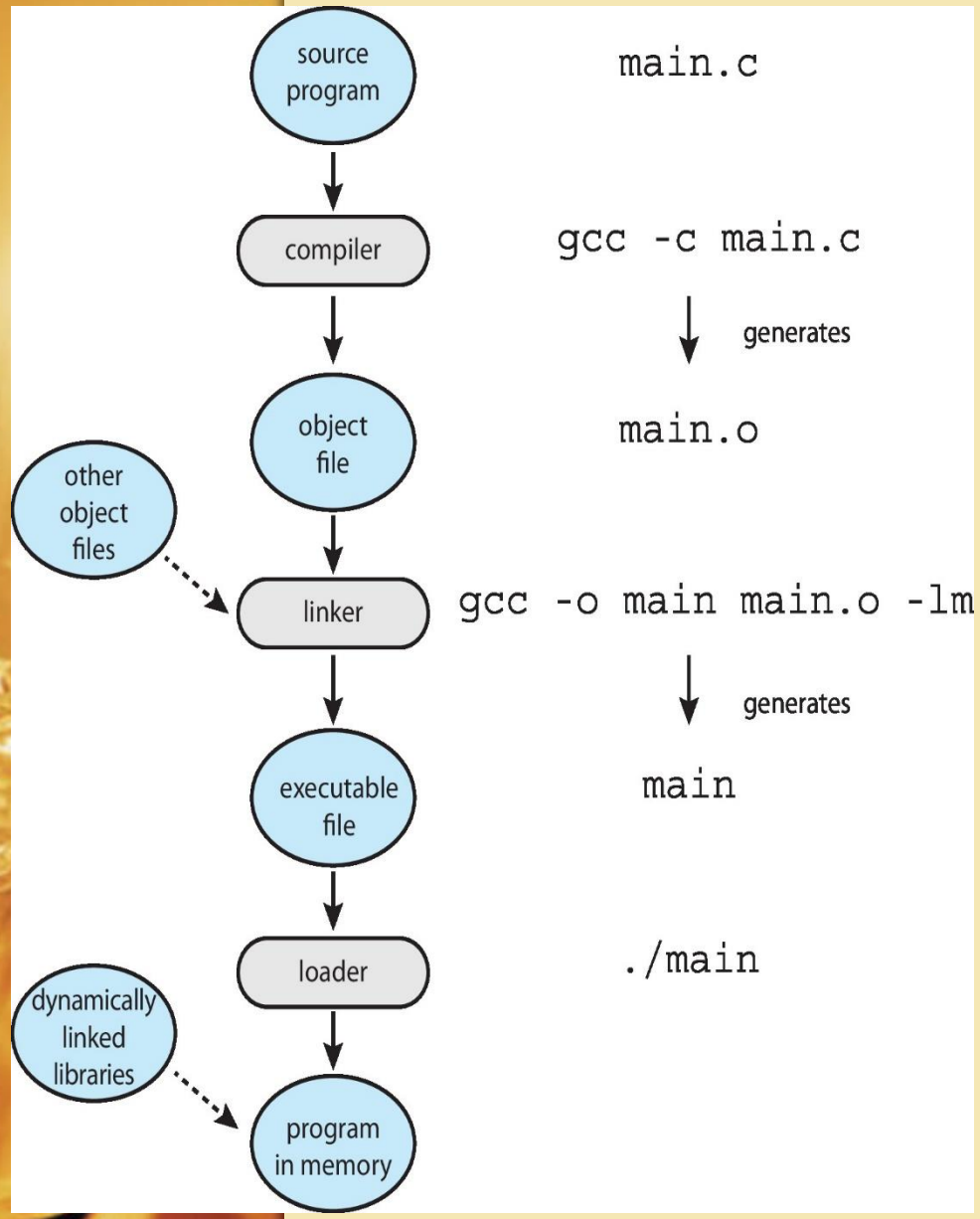
Chương trình hệ thống

- Cập nhật File
  - Trình soạn thảo văn bản giúp tạo và cập nhật tập tin
  - Các lệnh đặc biệt để tìm kiếm nội dung của các tập tin hoặc thực hiện chuyển đổi văn bản
- Ngôn ngữ lập trình – trình biên dịch, trình gỡ rối được cung cấp
- Chương trình tải và thực thi – trình nạp, trình liên kết, trình gỡ rối dùng cho ngôn ngữ bậc cao và ngôn ngữ máy
- Liên lạc - Cung cấp cơ chế cho việc tạo ra các kết nối ảo giữa các tiến trình, người dùng, và hệ thống máy tính
  - Cho phép người dùng gửi tin nhắn đến một màn hình của người khác, duyệt web, gửi mail, đăng nhập từ xa, chuyển các tập tin từ một máy tới máy khác

Chương trình hệ thống

- Dịch vụ nền
  - Chạy lúc khởi động
    - Một số chạy tại lúc hệ thống khởi động và lúc hệ thống kết thúc
    - Một số chạy từ lúc khởi động đến lúc tắt máy
  - Cung cấp các tiện ích như kiểm tra đĩa, điều phối tiến trình, báo lỗi đăng nhập, in ấn
  - Chạy trong chế độ người dùng, không chạy trong chế độ hạt nhân
- Chương trình ứng dụng
  - Không thuộc về hệ thống
  - Chạy bởi người dùng
  - Không được coi là một phần của hệ điều hành
  - Được khởi động bằng lệnh, chuột, cảm ứng tay

6. Trình liên kết và trình nạp



- Mã nguồn được biên dịch thành tập tin được thiết kế để có thể nạp vào bất cứ vị trí bộ nhớ vật lý nào – gọi là tập tin tái định vị **(relocatable object file)**
- Trình liên kết **(Linker)** kết hợp các tập tin này thành một tập tin thực thi nhị phân
  - Cũng mang chúng vào các thư viện
- Chương trình nằm trên bộ nhớ thứ cấp dưới dạng tập tin thực thi nhị phân
- Phải được đưa vào bộ nhớ bởi trình nạp **(loader)** để được thực thi
  - Trình chuyển vị trí **(Relocation)** gán địa chỉ cuối cùng cho các phần của chương trình, và điều chỉnh mã và dữ liệu trong chương trình để khớp với các địa chỉ đó.
- Các hệ thống đa dụng hiện đại không liên kết các thư viện với các tập tin thực thi
  - Thay vào đó, các thư viện liên kết động (**dynamically linked libraries** ,trong Windows, DLL) được tải khi cần thiết, được chia sẻ bởi tất cả những người sử dụng cùng một phiên bản của cùng một thư viện đó (được tải một lần)
- Tập tin thực thi có định dạng tiêu chuẩn, vì vậy hệ điều hành biết cách tải và khởi động chúng

7. Thiết kế và thực thi hệ điều hành

- Cấu trúc bên trong của các hệ điều hành khác nhau có thể khác nhau
- Bắt đầu thiết kế bằng việc xác định mục tiêu và các chi tiết kỹ thuật
- Ảnh hưởng bởi sự lựa chọn phần cứng, loại hệ thống
- Mục tiêu người dùng **(User)** và mục tiêu hệ thống **(System)**
  - User goals – hệ điều hành nên thuận tiện để sử dụng, dễ dàng để tìm hiểu, đáng tin cậy, an toàn và nhanh chóng
  - System goals – hệ điều hành phải được dễ dàng để thiết kế, thực hiện và duy trì, cũng như linh hoạt, đáng tin cậy, không có lỗi, và hiệu quả

Thiết kế và thực thi hệ điều hành

- Nguyên lý quan trọng:

  Chính sách (Policy): Sẽ làm ***những gì***? **(WHAT)**
  Cơ chế (Mechanism): Làm chúng ***như thế nào***? **(HOW)**
- Cơ chế xác định làm như thế nào, chính sách quyết định sẽ làm những gì
- Tách rời chính sách từ cơ chế là nguyên tắc quan trọng, nó cho phép linh hoạt tối đa nếu các quyết định chính sách được thay đổi
- Xác định và thiết kế một hệ điều hành là nhiệm vụ rất sáng tạo của ngành kỹ thuật phần mềm

Thực thi

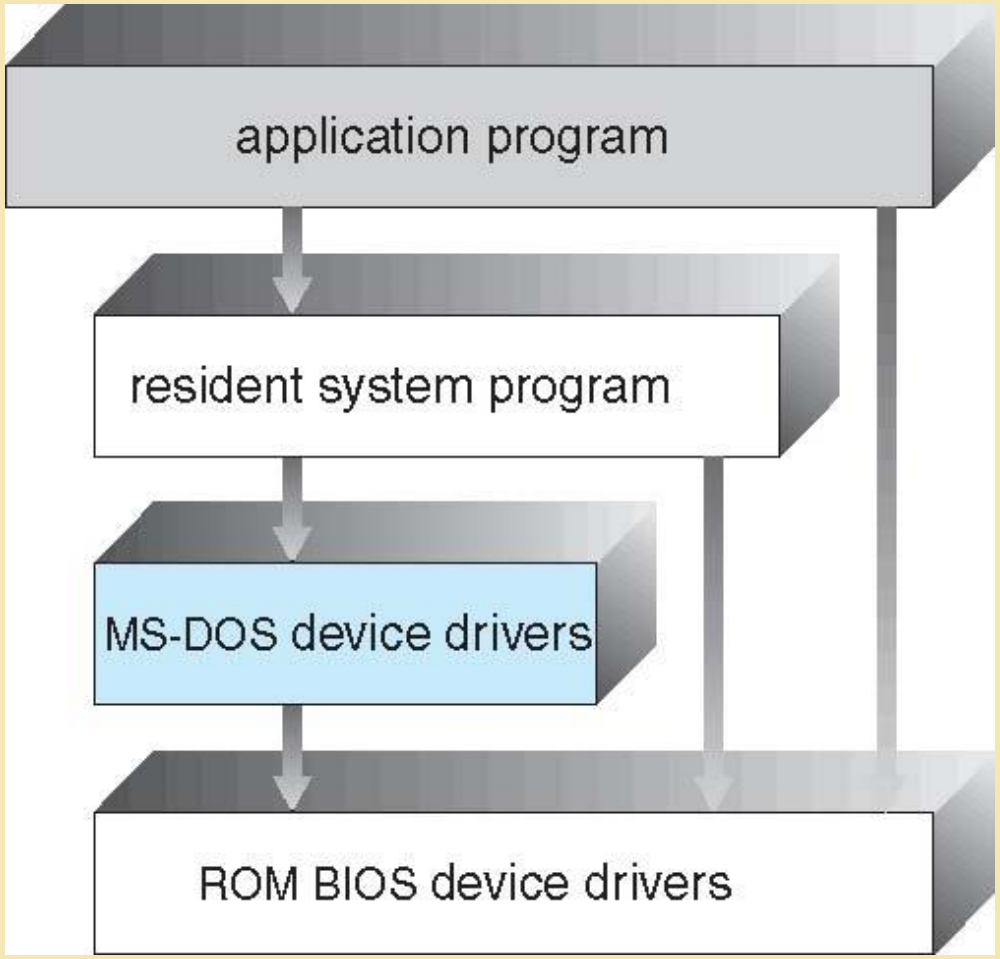
- Nhiều thay đổi theo thời gian
  - Các HĐH cũ viết bằng ngôn ngữ assembly
  - Kế tiếp viết bằng các ngôn ngữ như Algol, PL/1
  - Hiện nay viết bằng C, C++
- Thường kết hợp nhiều ngôn ngữ
  - Mức thấp nhất dùng assembly
  - Thân chính dùng C
  - Các chương trình hệ thống dùng C, C++, các ngôn ngữ kịch bản như PERL, Python, shell scripts
- Nhiều ngôn ngữ cấp cao dễ dàng hơn để mở **port** cho phần cứng
  - Nhưng chậm hơn
- **Emulation** cho phép HĐH chạy trên phần cứng non-native

8. Cấu trúc hệ điều hành

- Hệ điều hành đa dụng là một chương trình rất lớn
- Bao gồm các cấu trúc
  - Cấu trúc đơn giản – MS-DOS
  - Cấu trúc phức tạp – UNIX
  - Cấu trúc phân lớp – trừu tượng hóa
  - Cấu trúc Microkernel – Mach
  - Cấu trúc Module
  - Cấu trúc lai

Cấu trúc đơn giản – MS-DOS

- MS-DOS: được viết để cung cấp tối đa các chức năng trong 1 không gian nhớ ít nhất
  - Không được chia thành các đơn thể
  - Mặc dù MS-DOS có một số cấu trúc, nhưng giao diện và các cấp độ chức năng của nó không phân chia rõ ràng

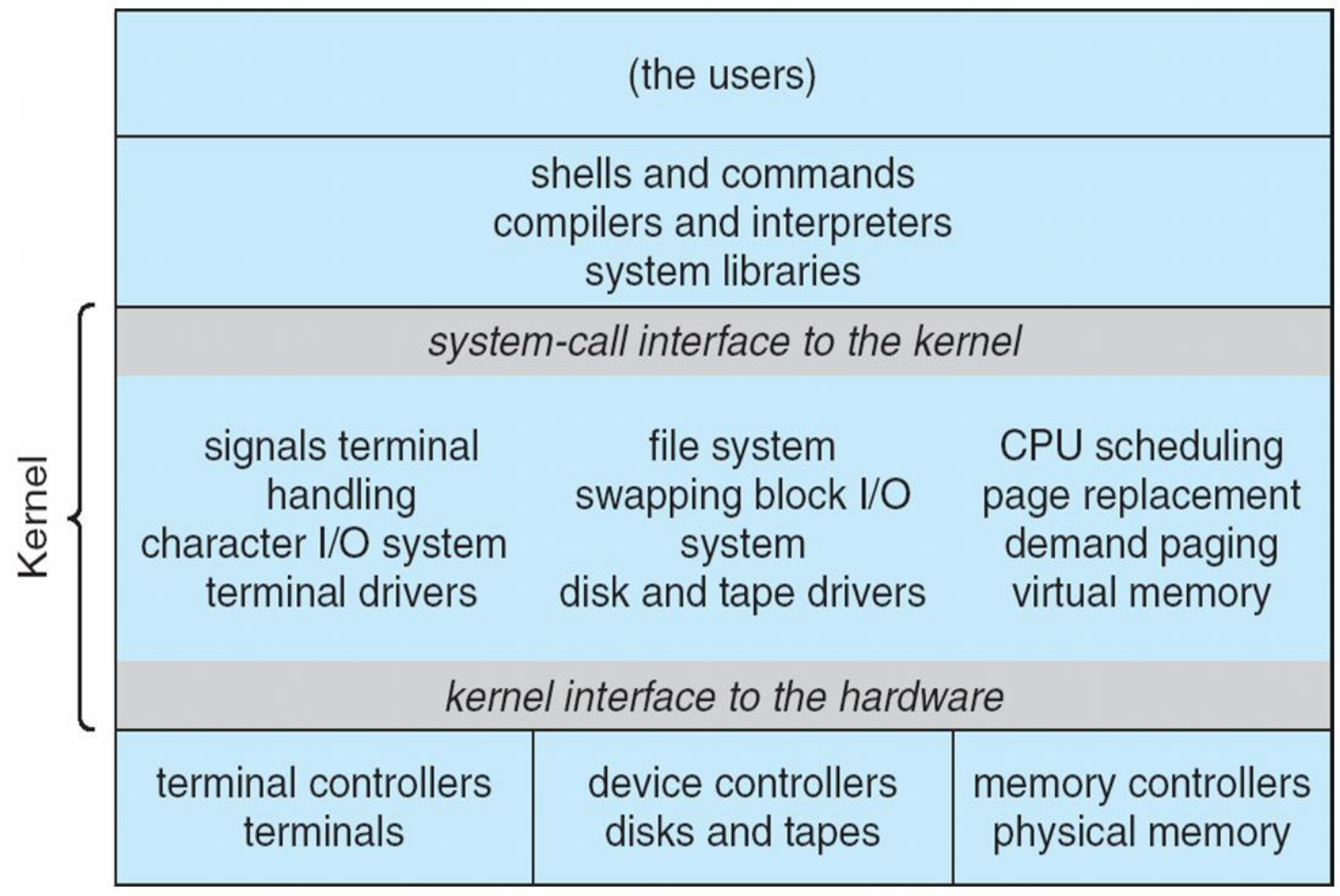


Cấu trúc không đơn giản - UNIX

- UNIX - giới hạn bởi chức năng phần cứng, bản gốc của hệ điều hành UNIX có cấu trúc hạn chế. Các hệ điều hành UNIX bao gồm hai phần riêng biệt:
  - Chương trình hệ thống
  - Nhân (Kernel)
    - Bao gồm tất cả mọi thứ bên dưới giao diện lời gọi hệ thống và bên trên phần cứng vật lý
    - Cung cấp hệ thống file, lập lịch CPU, quản lý bộ nhớ, và các chức năng khác

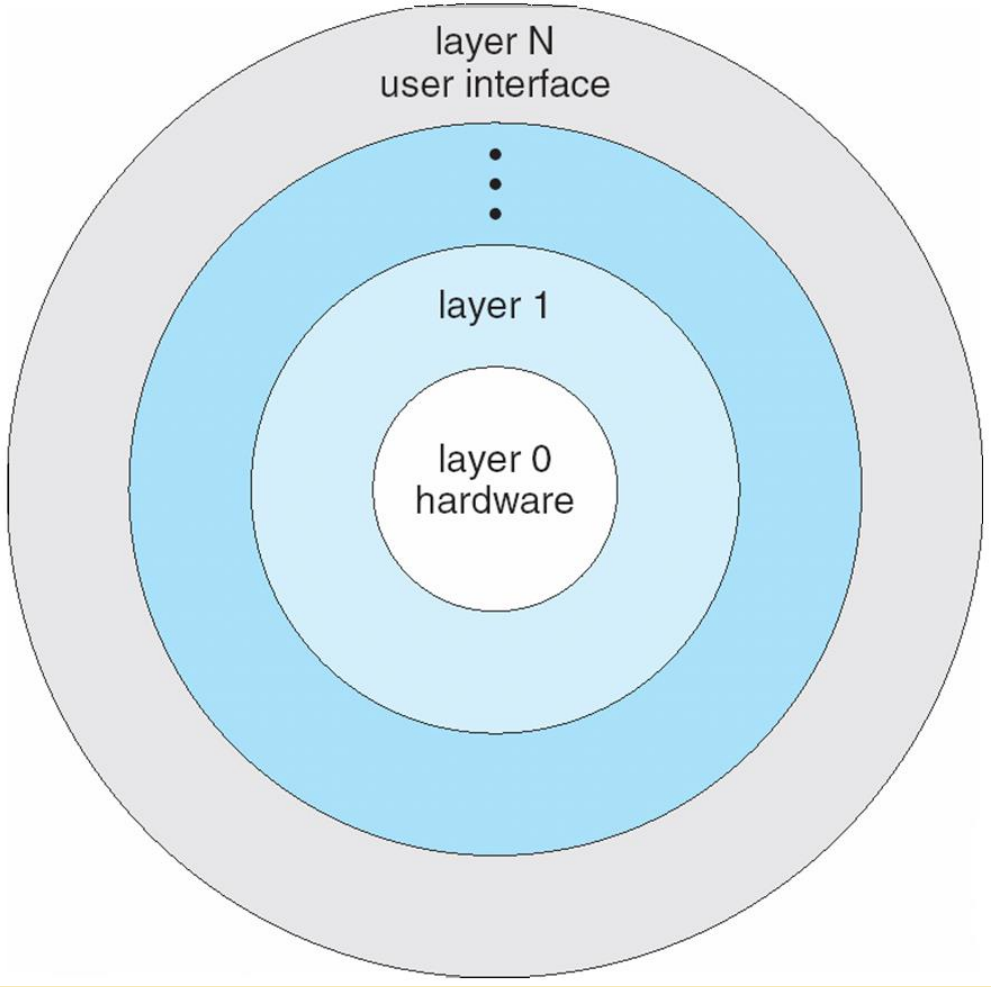
Cấu trúc hệ thống UNIX truyền thống

- Vượt ra cấu trúc đơn giản nhưng chưa phân lớp đầy đủ



Phương pháp tiếp cận phân lớp

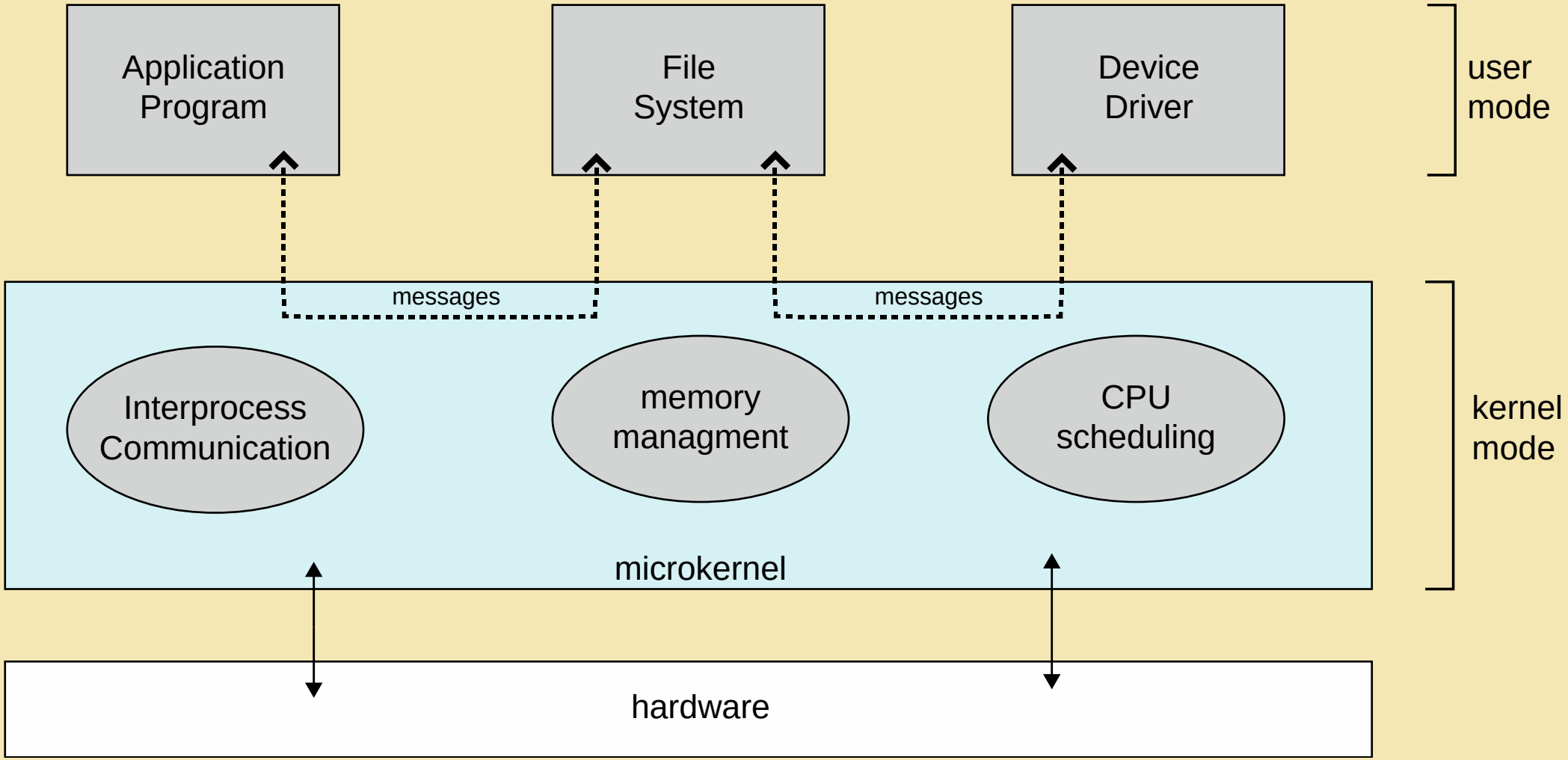
- Hệ điều hành được chia thành một số lớp (cấp độ), mỗi lớp được xây dựng dựa trên lớp thấp hơn. Lớp dưới cùng (layer 0) là phần cứng; lớp cao nhất (lớp N) là giao diện người dùng
- Với tính đơn thể hóa, mỗi lớp chỉ sử dụng các dịch vụ và chức năng của lớp thấp hơn



Cấu trúc Microkernel

- Di chuyển nhiều thành phần từ chế độ nhân sang chế độ người dùng
- HĐH Mach là ví dụ cho cấu trúc Microkernel
- Mac OS X kernel (Darwin) một phần dựa trên Mach
- Liên lạc giữa các module người dùng sử dụng cơ chế **message passing**
- Lợi ích:
  - Dễ dàng mở rộng
  - Dễ dàng chuyển HĐH sang một kiến trúc mới
  - Đáng tin cậy hơn (ít mã lệnh chạy trong kernel mode)
  - An toàn hơn
- Hạn chế
  - Việc liên lạc từ chế độ người dùng sang chế độ nhân tốn hiệu suất cao

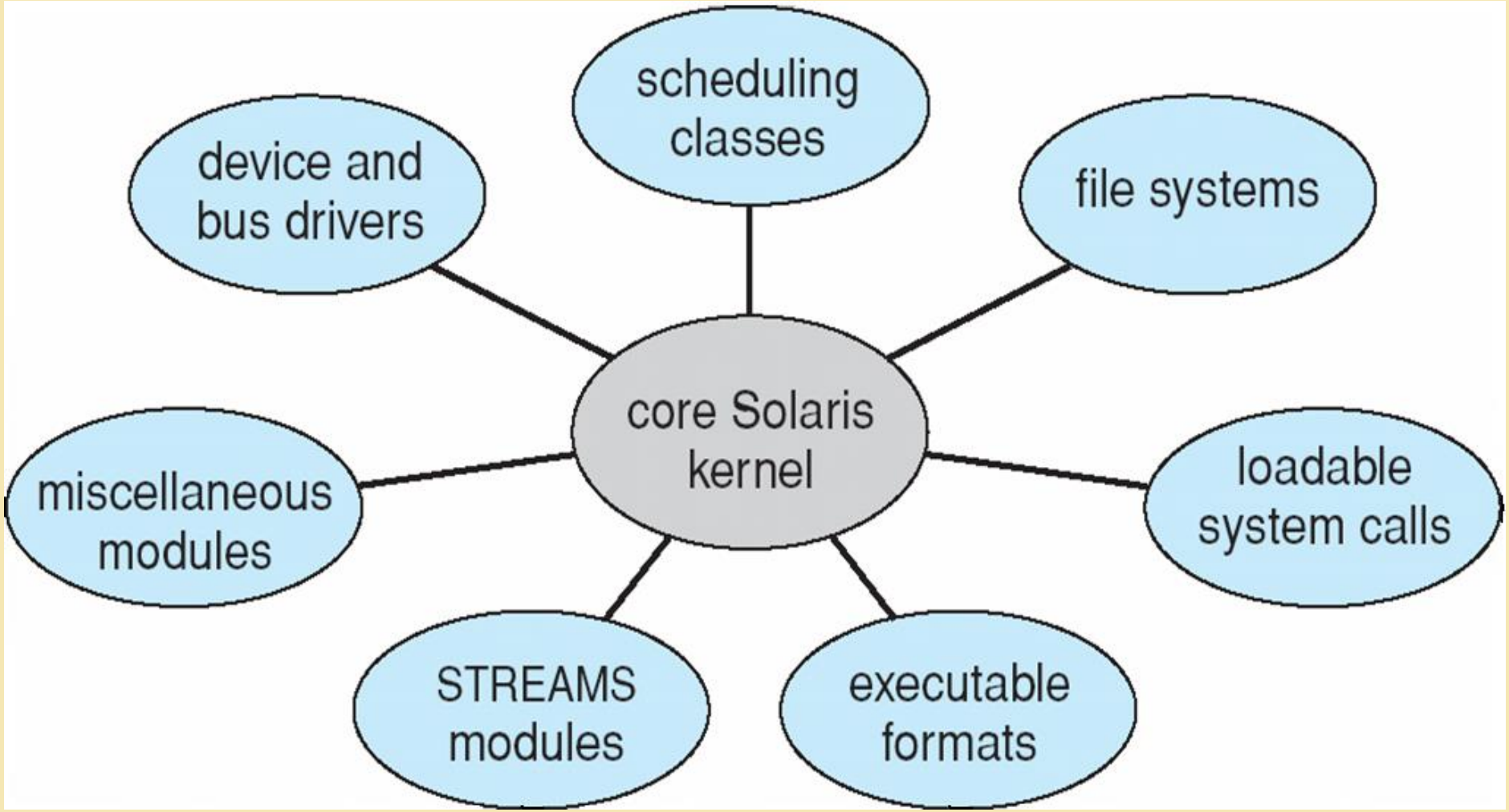
Cấu trúc Microkernel



Cấu trúc Module

- Nhiều HĐH hiện đại được thực thi theo cấu trúc module hóa hạt nhân
  - Sử dụng cách tiếp cận hướng đối tượng
  - Mỗi thành phần lõi riêng biệt
  - Mỗi thành phần lõi liên lạc với các thành phần khác qua giao diện
  - Mỗi thành phần lõi có thể được nạp thông qua nhân chính
- Giống cấu trúc phân lớp nhưng linh hoạt hơn
  - Linux, Solaris, …

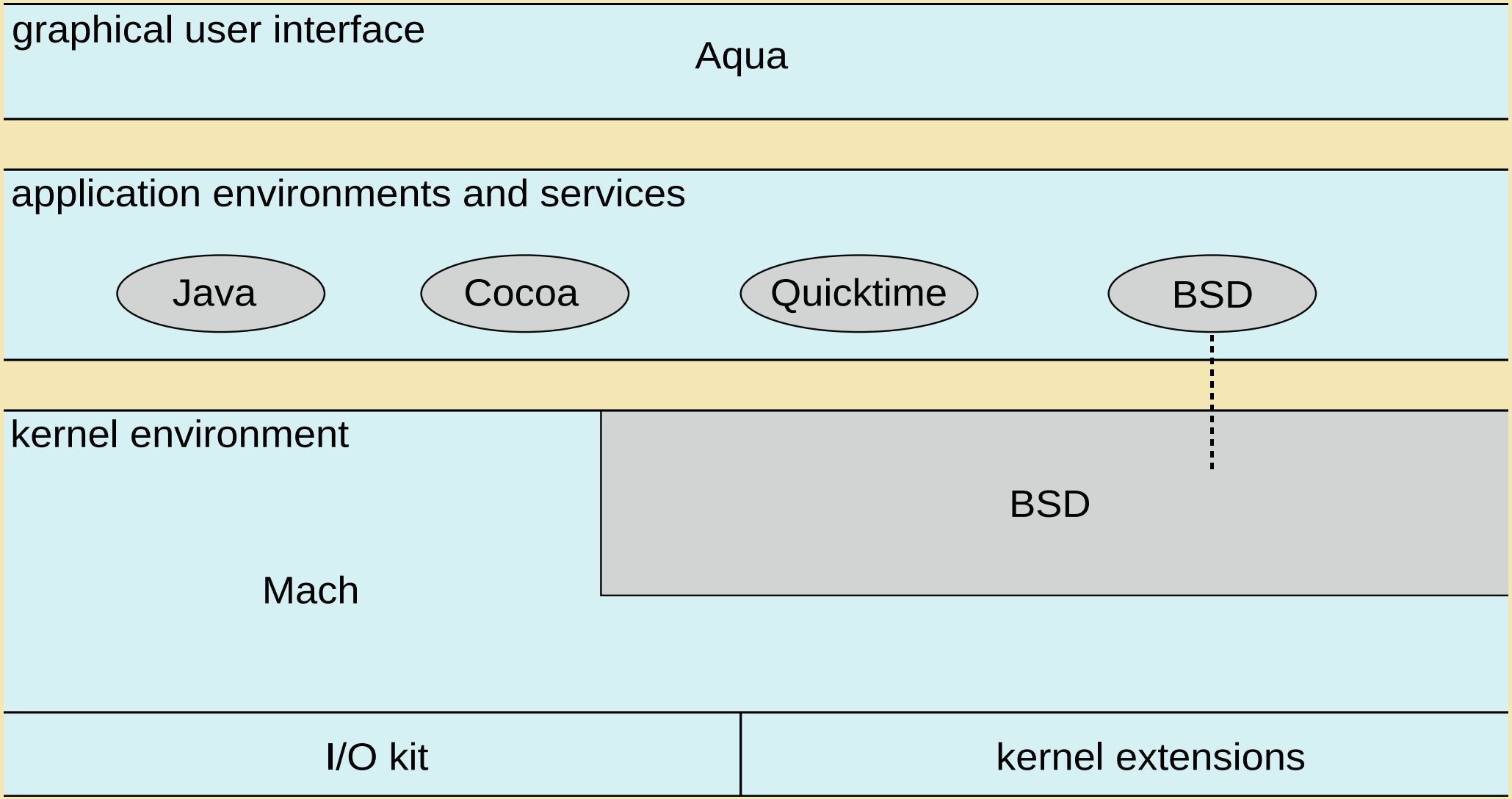
Cấu trúc Module - Solaris



Các hệ thống lai

- Hầu hết các hệ điều hành hiện đại thực sự không sử dụng một mô hình thuần túy
  - Mô hình lai sử dụng nhiều hướng tiếp cận để giải quyết tính hiệu quả, an ninh, tính khả dụng
  - Nhân của Linux và Solaris được thiết kế nguyên khối, cộng với các mô đun chức năng động
  - Windows hầu hết là nguyên khối, cộng với microkernel của các hệ thống phụ khác nhau – ***personalities***
- Apple Mac OS X là hệ thống lai, phân lớp, giao diện người dùng Aqua và môi trường lập trình Cocoa

Cấu trúc Mac OS X



iOS

- HĐH di động của Apple dùng cho iPhone, Ipad
  - Có cấu trúc dựa trên Mac OS X, cộng với 1 số chức năng bổ sung
  - Không chạy các ứng dụng OS X
  - **Cocoa Touch:** Objective-C API dùng việc phát triển các ứng dụng
  - **Media services:** đồ họa, âm thanh, video
  - **Core services:** điện toán đám mây, cơ sở dữ liệu
  - Core OS: dựa trên nhân HĐH Mac OS X

Cocoa Touch

Media Services

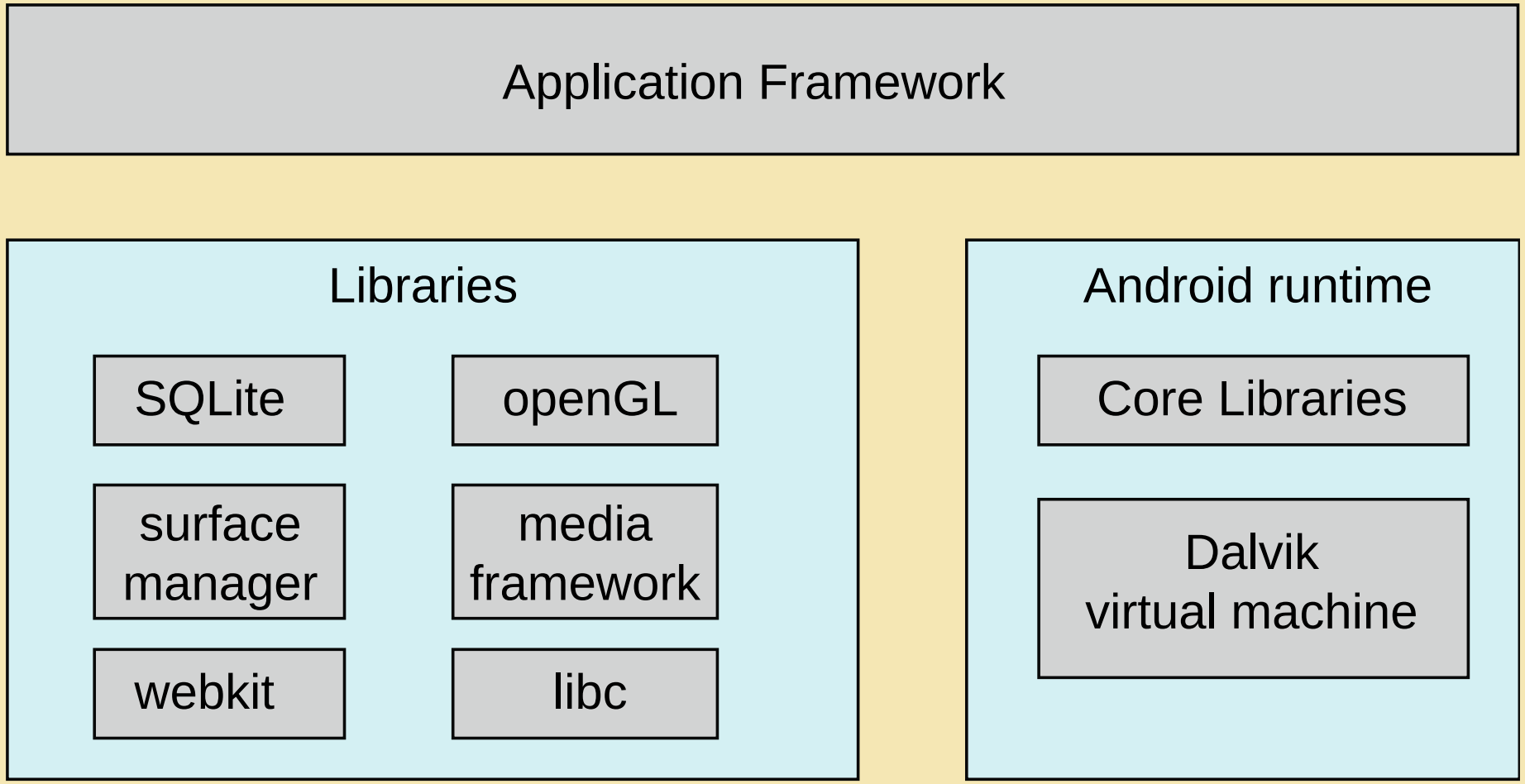
Core Services

Core OS

Android

- Được phát triển bởi Open Handset Alliance (chủ yếu là Google)
  - Mã nguồn mở
- Ngăn xếp phân lớp tương tự trong iOS
- Dựa trên nhân Linux nhưng có cập nhật
  - Cung cấp quản lý tiến trình, bộ nhớ, quản lý trình điều khiển thiết bị
  - Cộng thêm quản lý điện năng
- Môi trường thời gian chạy bao gồm bộ thư viện lõi và máy ảo Dalvik
  - Ứng dụng được phát triển trong Java cộng với API Android

Kiến trúc Android



Thắc mắc & Giải đáp

CÂU HỎI THẢO LUẬN

1. Mục đích của lời gọi hệ thống (System calls) là gì?
2. Nêu 5 hoạt động chính của hệ điều hành trong việc quản lý tiến trình?
3. Nêu 3 hoạt động chính của hệ điều hành trong việc quản lý bộ nhớ?
4. Mục đích của chương trình hệ thống (System programs) là gì?
5. Tại sao một số hệ thống lưu trữ hệ điều hành trong phần dẻo (firmware), trong khi một số khác lại lưu trữ trên đĩa?